

Département Informatique et Réseaux
Module : Conception des Systèmes d'Exploitation
Année universitaire 2025/2026

Rapport de Mini-Projet

Partie 1 : Ordonnanceur de Processus

**Partie 2 : Simulation de Bras Robotiques
Intelligents dans un Smart Factory**



Résumé

Ce rapport documente la réalisation d'un mini-projet dans le cadre du module *Conception des Systèmes d'Exploitation* à l'ENSEM. Il comporte deux parties indépendantes mais complémentaires.

La **Partie 1** consiste en l'implémentation d'un *ordonnanceur de processus* en langage C. Le programme lit un fichier de configuration décrivant un ensemble de processus (nom, date d'arrivée, durée CPU), puis simule trois politiques classiques d'ordonnancement : **FIFO** (*First In First Out*), **SJF** (*Shortest Job First*) et **Round Robin**. Les résultats sont affichés sous forme de diagramme de Gantt textuel accompagné de statistiques (temps d'attente, temps de séjour et leurs moyennes).

La **Partie 2** simule un *entrepôt intelligent* dans lequel plusieurs bras robotiques (clients multithreadés) partagent des outils communs via un serveur central de gestion des ressources. La communication repose sur des sockets TCP. Côté serveur, trois mécanismes combinés préviennent les interblocages : hiérarchie fixe des ressources, algorithme *wound-wait* et un thread *watchdog*. Un journal CSV horodaté trace l'ensemble des événements système.

Mots-clés : ordonnancement, FIFO, SJF, Round Robin, diagramme de Gantt, sockets TCP, threads, mutex, deadlock, wound-wait, watchdog, C.

Contents

Résumé	1
Introduction Générale	4
I Ordonnanceur de Processus	5
1 Analyse et Conception	6
1.1 Présentation du problème.....	6
1.2 Architecture générale.....	6
1.3 Structure de données.....	7
1.4 Format du fichier de configuration.....	8
1.5 Diagramme de Gantt.....	8
2 Implémentation	9
2.1 Point d'entrée principal.....	9
2.2 Parseur de configuration.....	10
2.3 Politique FIFO.....	10
2.4 Politique SJF.....	11
2.5 Politique Round Robin.....	12
2.6 Compilation et exécution.....	13
3 Résultats et Analyse	14
3.1 Simulation avec l'exemple de configuration.....	14
3.1.1 Résultats FIFO.....	14
3.1.2 Résultats SJF.....	14
3.1.3 Résultats Round Robin (quantum = 2).....	15
3.2 Comparaison des politiques.....	15
II Simulation de Bras Robotiques dans un Smart Factory	16
4 Analyse et Conception	17
4.1 Présentation du problème.....	17
4.2 Architecture du système.....	17
4.3 Protocole de communication.....	18
5 Implémentation	19

5.1	En-tête commun.....	19
5.2	Serveur – Gestionnaire d'outils.....	19
5.2.1	Structure de données des outils.....	19
5.2.2	Mécanismes de prévention des interblocages.....	20
5.2.3	Journalisation CSV.....	21
5.3	Client – Bras Robotique Multithreadé.....	22
5.4	Compilation et exécution.....	23
6	Résultats et Analyse	24
6.1	Extrait du journal de logs.....	24
6.2	Preuve d'absence de deadlock.....	24
	Conclusion Générale	26

Introduction Générale

La maîtrise des mécanismes internes des systèmes d'exploitation est une compétence fondamentale pour tout ingénieur en informatique. L'ordonnancement du processeur et la gestion concurrente des ressources partagées constituent deux piliers essentiels de cette discipline.

Ce mini-projet, réalisé dans le cadre du module *Conception des Systèmes d'Exploitation*, vise à mettre en pratique ces concepts à travers deux applications concrètes développées en langage C.

La première application, l'ordonnanceur de processus, illustre comment un système d'exploitation décide de l'ordre d'exécution des processus en fonction de différents critères. Elle permet de comparer quantitativement les performances des politiques FIFO, SJF et Round Robin sur un ensemble de processus défini dans un fichier de configuration.

La seconde application, la simulation d'un entrepôt intelligent, constitue une adaptation du célèbre problème du dîner des philosophes dans un contexte industriel réaliste. Des bras robotiques, représentés par des processus clients multithreadés, s'y disputent des outils partagés en communiquant avec un serveur central via le protocole TCP/IP. La problématique des interblocages y est traitée de façon rigoureuse par plusieurs mécanismes complémentaires.

Ce rapport présente l'analyse, la conception et l'implémentation détaillée de ces deux projets, ainsi que les résultats obtenus et les pistes d'amélioration.

Part I

Ordonnanceur de Processus

Chap : 1 Analyse et Conception

1.1 Présentation du problème

Un ordonnanceur de processus est le composant du noyau d'un système d'exploitation chargé de décider quel processus s'exécutera sur le CPU à un instant donné. Ce choix repose sur une **politique d'ordonnancement** qui détermine l'ordre et la durée d'attribution du processeur à chaque processus.

Les critères de qualité d'un ordonnanceur sont généralement :

- **Temps d'attente** (T_a) : durée entre l'arrivée d'un processus et le début de son exécution (ou le cumul des périodes d'attente).
- **Temps de séjour** (T_s) : durée totale entre l'arrivée et la terminaison du processus. On a $T_s = T_{fin} - T_{arrive}$.
- **Temps de réponse** : temps entre l'arrivée et la première fois où le processus obtient le CPU.

* Relation fondamentale

$$T_s = T_{fin} - T_{arrive} \quad \text{et} \quad T_a = T_s - T_{dure'e}$$

L'objectif d'un bon algorithme est de minimiser les moyennes $\bar{T}_a$ et $\bar{T}_s$.

1.2 Architecture générale

L'architecture logicielle de l'ordonnanceur suit un principe de **séparation des responsabilités** :

- `src/main.c` — point d'entrée, lecture du fichier de configuration, menu interactif et dispatching vers la politique choisie.
- `src/parser.c` — parseur du fichier de configuration.
- `include/processus.h` — structure de données `Processus`.
- `include/scheduler.h` — définition du type pointeur de fonction `policy_func`.

- ▷ `policies/fifo.c`, ▷ `policies/sjf.c`, ▷ `policies/rr.c` — implémentations des trois politiques, chacune dans un fichier indépendant.
- ▷ `config/config.txt` — fichier de configuration des processus.
- ▷ `Makefile` — script de compilation automatisée.

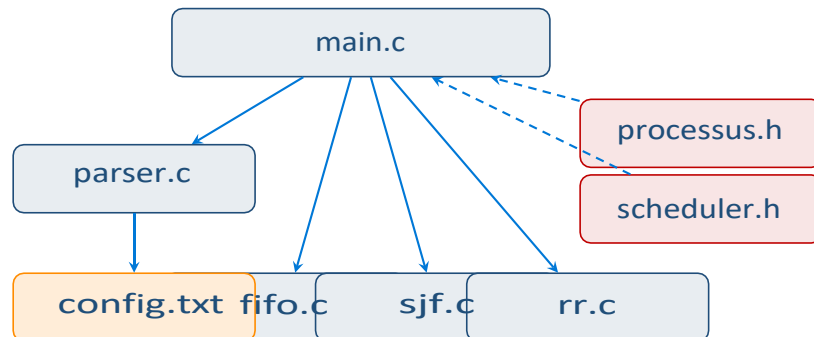


Figure 1.1: Architecture modulaire de l'ordonnanceur

1.3 Structure de données

La structure `Processus`, définie dans ▷ `include/processus.h`, encapsule toutes les informations nécessaires à la simulation :

```

1 typedef struct {
2     char nom[10];    // Identifiant textuel (ex. P1, P2...)
3     int arrivee;     // Date d'arrivée dans la file
4     int duree;       // Durée totale CPU requise
5     int restant;     // Temps CPU restant (pour Round Robin)
6
7     int debut;       // Instant de premier démarrage
8     int fin;         // Instant de terminaison
9 } Processus;

```

Listing 1.1: Définition de la structure `Processus`

L'**extensibilité** de la conception est assurée par la définition d'un pointeur de fonction dans ▷ `include/scheduler.h` :

```

1 typedef void (*policy_func)(Processus*, int);
2
3 typedef struct {
4     char nom[50];
5     policy_func fonction;
6 } Policy;

```

Listing 1.2: Type pointeur de fonction de politique

Ce mécanisme permettrait de charger dynamiquement (via `dlopen/dlsym` sous Linux) de nouvelles politiques placées dans le répertoire `policies/` sans recompiler le programme principal.

1.4 Format du fichier de configuration

Le parseur implémenté dans `src/parser.c` lit un fichier texte ligne par ligne. Chaque processus est décrit sur une ligne selon le format :

```
<Nom> <Arrivée> <Durée>
```

Les lignes commençant par `#` sont traitées comme des commentaires et les lignes vides sont ignorées. Exemple de fichier de configuration :

```
1 # test avancé
2
3 P1 0 5
4
5 # autre processus
6 P2 1 3
7 P3 2 2
```

Listing 1.3: Exemple de fichier config/config.txt

1.5 Diagramme de Gantt

Pour chacun des trois processus de l'exemple (P1 arrivée=0 durée=5, P2 arrivée=1 durée=3, P3 arrivée=2 durée=2), les diagrammes de Gantt attendus pour les trois politiques sont les suivants :

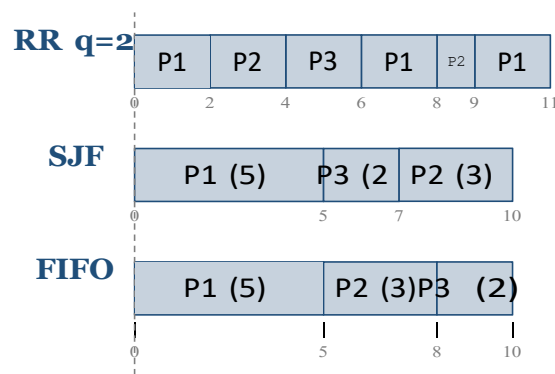


Figure 1.2: Diagrammes de Gantt pour les trois politiques d'ordonnancement

Chap : 2 Implémentation

2.1 Point d'entrée principal

Le fichier `src/main.c` orchestre l'ensemble du programme. Il charge les processus depuis le fichier de configuration, affiche le menu interactif, puis dispatch l'exécution vers la politique choisie par l'utilisateur.

```
1 int main() {
2     Processus tab[100];
3     int n, choix;
4
5     n = lire_processus("config/config.txt", tab);
6     if (n == 0) {
7         printf("Erreur : aucun processus charge\n");
8         return 1;
9     }
10
11     printf("Choisir l'algorithme :\n");
12     printf("1. FIFO  2. SJF  3. Round Robin\n");
13     scanf("%d", &choix);
14
15     switch (choix) {
16         case 1: fifo(tab, n); break;
17         case 2: sjf(tab, n); break;
18         case 3: {
19             int quantum;
20             printf("Entrer le quantum : ");
21             scanf("%d", &quantum);
22             round_robin(tab, n, quantum);
23             break;
24         }
25         default: printf("Choix invalide\n");
26     }
27     return 0;
28 }
```

Listing 2.1: Programme principal – main.c

2.2 Parseur de configuration

```
1 int lire_processus(const char *filename, Processus tab[])
2 { FILE *f = fopen(filename, "r");
3   if (!f) return 0;
4
5   char ligne[100];
6   int i = 0;
7
8   while (fgets(ligne, sizeof(ligne), f)) {
9     // Ignorer commentaires et lignes vides
10    if (ligne[0] == '#' || ligne[0] == '\n')
11      continue;
12
13    sscanf(ligne, "%s %d %d",
14           tab[i].nom, &tab[i].arrivee, &tab[i].duree);
15    tab[i].restant = tab[i].duree;
16    i++;
17  }
18
19  fclose(f);
20  return i;
21 }
```

Listing 2.2: Parseur de fichier – src/parser.c

2.3 Politique FIFO

L'algorithme FIFO (*First In First Out*) exécute les processus dans l'ordre de leur arrivée, sans préemption. Si le CPU est libre avant l'arrivée du prochain processus, le temps est avancé à la date d'arrivée de ce dernier.

```
1 void fifo(Processus tab[], int n) {
2   int temps = 0;
3
4   printf("\n===== FIFO =====\n\nGantt : \n");
5
6   for (int i = 0; i < n; i++) {
7     // Avancer le temps si nécessaire
8     if (temps < tab[i].arrivee)
9       temps = tab[i].arrivee;
10
11     tab[i].debut = temps;
12
13     for (int t = 0; t < tab[i].duree; t++) {
14       printf("| %s ", tab[i].nom);
15       temps++;
16     }
17     tab[i].fin = temps;
18   }
19   printf("\n\n");
20 }
```

```

20
21 // Affichage des statistiques
22 float total_attente = 0, total_sejour = 0;
23 printf("Processus | Arr | Dur | Deb | Fin | Att | Sej\n");
24
25 for (int i = 0; i < n; i++) {
26     int sejour = tab[i].fin - tab[i].arrivee;
27     int attente = sejour - tab[i].duree;
28     total_attente += attente;
29     total_sejour += sejour;
30     printf("%s\t %d\t %d\t %d\t %d\t %d\t %d\n",
31           tab[i].nom, tab[i].arrivee, tab[i].duree,
32           tab[i].debut, tab[i].fin, attente, sejour);
33 }
34 printf("\nMoyenne attente = %.2f\n", total_attente / n);
35 printf("Moyenne sejour = %.2f\n", total_sejour / n);
36 }

```

Listing 2.3: Implémentation FIFO – policies/fifo.c

2.4 Politique SJF

L'algorithme SJF (*Shortest Job First*) choisit à chaque instant le processus **disponible** (arrivé et non terminé) ayant la plus courte durée CPU. Cette version est **non préemptive** : une fois un processus lancé, il s'exécute jusqu'à sa terminaison.

```

1 void sjf(Processus tab[], int n) {
2     int termine[n];
3     for (int i = 0; i < n; i++) termine[i] = 0;
4
5     int temps = 0, fini = 0;
6     printf("\n==== SJF =====\n\nGantt : \n");
7
8     while (fini < n) {
9         int min = -1;
10
11         // Chercher le processus prêt le plus court
12         for (int i = 0; i < n; i++) {
13             if (!termine[i] && tab[i].arrivee <= temps) {
14                 if (min == -1 || tab[i].duree < tab[min].duree)
15                     min = i;
16             }
17         }
18
19         if (min == -1) { temps++; continue; } // CPU inactif
20
21         tab[min].debut = temps;
22         for (int t = 0; t < tab[min].duree; t++)
23             { printf("| %s ", tab[min].nom);
24               temps++;
25         }

```

```

26     tab[min].fin = temps;
27     termine[min] = 1;
28     fini++;
29 }
30 printf("|\\n\\n");
31 // ... (statistiques identiques à FIFO)
32 }

```

Listing 2.4: Implémentation SJF – politiques/sjf.c

2.5 Politique Round Robin

Le Round Robin partage équitablement le CPU entre les processus en leur allouant successivement des tranches de temps de durée fixe (*quantum*). Si un processus ne termine pas pendant son quantum, il retourne en fin de file.

```

1 void round_robin(Processus tab[], int n, int quantum) {
2     int temps = 0, reste = n;
3     printf("\\n==== ROUND ROBIN (q=%d) ====\\n\\nGantt :\\n",
4           quantum);
5
6     for (int i = 0; i < n; i++)
7         { tab[i].restant = tab[i].
8           duree; tab[i].debut = -1;
9         }
10
11     while (reste > 0) {
12         int actif = 0;
13         for (int i = 0; i < n; i++) {
14             if (tab[i].restant > 0 && tab[i].arrivee <= temps)
15                 { actif = 1;
16                   if (tab[i].debut == -1)
17                       tab[i].debut = temps;
18
19                   int exec = (tab[i].restant < quantum)
20                             ? tab[i].restant : quantum;
21                   for (int t = 0; t < exec; t++)
22                       { printf("| %s ", tab[i].
23                         nom); temps++;
24                         }
25                   tab[i].restant -= exec;
26
27                   if (tab[i].restant == 0)
28                       { tab[i].fin = temps;
29                         reste--;
30                       }
31                 }
32             if (!actif) temps++; // Avancer si aucun processus prêt
33         }
34         // ... (affichage Gantt et statistiques)

```

35 }

Listing 2.5: Implémentation Round Robin – policies/rr.c

2.6 Compilation et exécution

```
1 CC=gcc
2
3 all:
4     $(CC) src/main.c src/parser.c policies/*.c -o ordonnanceur
5
6 run:
7     ./ordonnanceur
```

Listing 2.6: Makefile de l'ordonnanceur

La commande `make` compile l'ensemble des sources en une seule passe ; `make run` lance immédiatement l'exécutable.

Chap : 3 Résultats et Analyse

3.1 Simulation avec l'exemple de configuration

En utilisant le fichier [config/config.txt](#) fourni (P1:0,5 — P2:1,3 — P3:2,2), les résultats obtenus pour chaque politique sont les suivants.

3.1.1 Résultats FIFO

Table 3.1: Statistiques FIFO

Processus	Arrivée	Durée	Début	Fin	Attente	Séjour
P1	0	5	0	5	0	5
P2	1	3	5	8	4	7
P3	2	2	8	10	6	8
Moyenne					3,33	6,67

3.1.2 Résultats SJF

Table 3.2: Statistiques SJF

Processus	Début	Fin	Attente	Séjour
P1	0	5	0	5
P3	5	7	3	5
P2	7	10	6	9
Moyenne			3,00	6,33

3.1.3 Résultats Round Robin (quantum = 2)

Table 3.3: Statistiques Round Robin (q=2)

Processus	Arrivée	Durée	Fin	Attente	Séjour
P1	0	5	11	6	11
P2	1	3	9	5	8
P3	2	2	6	2	4
Moyenne				4,33	7,67

3.2 Comparaison des politiques

Politique	T_a^- moyen	T_s^- moyen
FIFO	3,33	6,67
SJF	3,00	6,33
Round Robin (q=2)	4,33	7,67

L'algorithme **SJF** donne les meilleures performances sur cet exemple car il privilégie les processus courts qui arrivent rapidement. En contrepartie, il peut provoquer une *famine* des processus longs dans un système à charge variable. **FIFO** est simple et prévisible mais pas optimal. **Round Robin** offre la meilleure équité au détriment du débit global.

Part II

Simulation de Bras Robotiques dans un Smart Factory

Chap : 4 Analyse et Conception

4.1 Présentation du problème

Ce projet simule un **entrepôt intelligent** où plusieurs bras robotiques doivent coopérer pour réaliser des tâches d'assemblage. Chaque bras a besoin d'**outils partagés** (tournevis, pince, soudeuse...) dont le nombre est limité.

Ce scénario est une adaptation industrielle du **problème du dîner des philosophes** : chaque bras (philosophe) doit obtenir deux outils (fourchettes) avant de travailler (manger). Si tous les bras tentent de prendre le même outil simultanément, un interblocage (*deadlock*) peut survenir.

4.2 Architecture du système

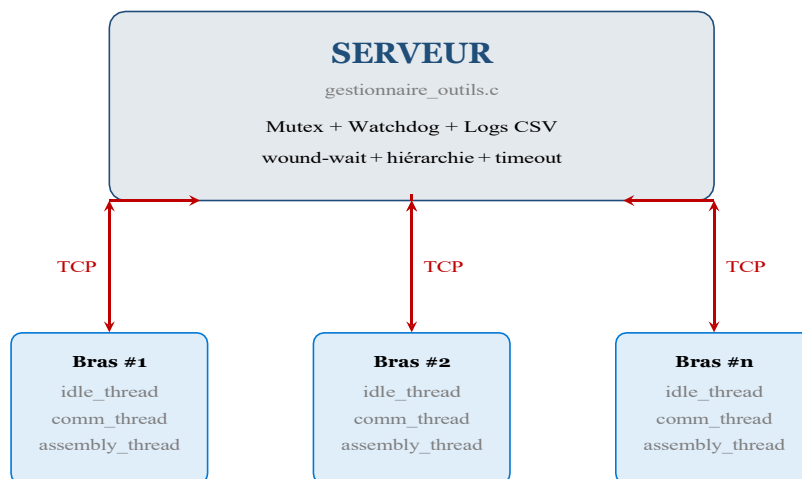


Figure 4.1: Architecture client-serveur de l'entrepôt intelligent

4.3 Protocole de communication

Les échanges entre clients et serveur utilisent un **protocole texte** simple, chaque message étant une ligne terminée par `\n`.

Table 4.1: Commandes du protocole (client → serveur)

Commande	Exemple	Signification
DEMANDE_OUTIL <id>	DEMANDE_OUTIL 2	Demande l'outil n°2
LIBERATION_OUTIL <id>	LIBERATION_OUTIL 2	Libère l'outil n°2
DEMANDE_DEUX_OUTILS <id1> <id2>	DEMANDE_DEUX_OUTILS 0 1	Demande simultanée

Table 4.2: Réponses du protocole (serveur → client)

Réponse	Signification
OK	Demande acceptée
REFUS	Outils non disponibles (réessayer)
LIBERE_OK	Libération confirmée
ID_INVALIDE	Identifiant d'outil incorrect
ERREUR_COMMANDE	Message non reconnu

Chap : 5 Implémentation

5.1 En-tête commun

Le fichier `include/commun.h` centralise toutes les constantes et macros partagées entre le client et le serveur :

```
1 #ifndef COMMUN_H
2 #define COMMUN_H
3
4 #define PORT      8888
5 #define NB_OUTILS 3
6
7 // Commandes
8 #define CMD_DEMANDE_OUTIL      "DEMANDE_OUTIL"
9 #define CMD_LIBERATION_OUTIL  "LIBERATION_OUTIL"
10 #define CMD_DEMANDE_DEUX_OUTILS "DEMANDE_DEUX_OUTILS"
11
12 // Réponses
13 #define REP_OK      "OK"
14 #define REP_REFUS   "REFUS"
15 #define REP_ID_INVALIDE "ID_INVALIDE"
16 #define REP_LIBERE_OK "LIBERE_OK"
17
18 #endif
```

Listing 5.1: Définitions communes – `include/commun.h`

5.2 Serveur – Gestionnaire d'outils

5.2.1 Structure de données des outils

```
1 #define TIMEOUT_SEC 10
2
3 typedef struct {
4     int    allocated;    // 1 = occupé, 0 = libre
5     int    owner_id;     // ID du client propriétaire
```

```

6     time_t alloc_time; // Horodatage d'allocation
7 } OutilInfo;
8
9 OutilInfo outils[NB_OUTILS];
10 HANDLE mutex; // Protège l'accès aux outils
11 FILE* log_file = NULL;

```

Listing 5.2: Structure OutilInfo et variables globales

5.2.2 Mécanismes de prévention des interblocages

Trois stratégies complémentaires sont combinées.

Stratégie 1 : Hiérarchie fixe des ressources

Pour toute demande de deux outils, le serveur trie les identifiants en ordre croissant avant d'allouer, garantissant que tous les clients demandent les ressources dans le même ordre et éliminant les cycles d'attente circulaires.

```

1 int allouer_deux_outils(int id1, int id2, int client_id) {
2     // Tri pour ordre fixe (hiérarchie)
3     int a = id1, b = id2;
4     if (a > b) { int tmp = a; a = b; b = tmp; }
5     // ...
6 }

```

Listing 5.3: Tri imposé pour DEMANDE_DEUX_OUTILS

Stratégie 2 : Algorithme Wound-Wait

Chaque client reçoit un identifiant unique incrémental. Un client d'ID plus petit est considéré comme « plus âgé » et prioritaire.

```

1 int allouer_outil(int id, int client_id) {
2     if (!outils[id].allocated) {
3         // Outil libre -> allocation directe
4         outils[id].allocated = 1;
5         outils[id].owner_id = client_id;
6         outils[id].alloc_time = time(NULL);
7         log_message("ALLOC", "Client %d a obtenu l'outil %d",
8                     client_id, id);
9         return 1;
10    } else {
11        // Outil occupé : wound-wait
12        if (client_id < outils[id].owner_id) {
13            // Client plus âgé -> WOUND (il blesse le plus jeune)
14            log_message("WOUND", "Client %d blesse client %d sur l'outil %d",
15                        client_id, outils[id].owner_id, id);
16            outils[id].owner_id = client_id;
17            outils[id].alloc_time = time(NULL);
18            return 1;

```

```

19     } else {
20         // Client plus jeune -> WAIT (refus, doit réessayer)
21         log_message("WAIT", "Client %d attend l'outil %d
22                     (detenu par %d)",
23                     client_id, id, outils[id].owner_id);
24         return 0;
25     }
26 }

```

Listing 5.4: Algorithme wound-wait dans `allouer_outil()`

Stratégie 3 : Thread Watchdog (timeout)

Un thread dédié surveille en permanence l'état des outils. Tout outil détenu depuis plus de `TIMEOUT_SEC` secondes est automatiquement libéré, ce qui protège contre les clients plantés sans avoir libéré leurs ressources.

```

1  DWORD WINAPI watchdog(LPVOID param) {
2      while (1) {
3          Sleep(5000); // vérification toutes les 5 secondes
4          WaitForSingleObject(mutex, INFINITE);
5          time_t now = time(NULL);
6          for (int i = 0; i < NB_OUTILS; i++) {
7              if (outils[i].allocated &&
8                  (now - outils[i].alloc_time) > TIMEOUT_SEC)
9                  { log_message("WATCHDOG",
10                               "Timeout : liberation forcee outil %d (client %d
11                               )",
12                               i, outils[i].owner_id);
13                    outils[i].allocated = 0;
14                    outils[i].owner_id = -1;
15                }
16            }
17            ReleaseMutex(mutex);
18        }
19        return 0;
20    }

```

Listing 5.5: Thread watchdog de libération automatique

5.2.3 Journalisation CSV

```

1  void log_message(const char* level, const char* format, ...)
2  { time_t now; time(&now);
3    struct tm* ti = localtime(&now);
4    char timestamp[64];
5    strftime(timestamp, sizeof(timestamp), "%Y-%m-%dT%H:%M:%S", ti
6              );
7
7    char buffer[1024];
8    va_list args;

```

```

9  va_start(args, format);
10 vsnprintf(buffer, sizeof(buffer), format, args);
11 va_end(args);
12
13 printf("[%s] [%s] %s\n", timestamp, level, buffer);
14 if (log_file)
15     fprintf(log_file, "%s;%s;%s\n", timestamp, level, buffer);
16 }

```

Listing 5.6: Fonction de journalisation horodatée

5.3 Client – Bras Robotique Multithreadé

Chaque bras robotique est un processus indépendant comportant **trois threads** qui communiquent via des files d'attente protégées par des sections critiques Windows ([CRITICAL_SECTION](#)).

Table 5.1: Rôle des trois threads du client

Thread	Rôle
idle_thread	Simule une phase de réflexion (attente aléatoire 3–10 s), puis génère aléatoirement une demande d'un ou deux outils et la place dans la file des requêtes.
comm_thread	Consomme les requêtes de la file, les envoie au serveur via la socket TCP, reçoit les réponses et les place dans la file des réponses.
assembly_thread	Consomme les réponses OK : simule le travail d'assemblage (Sleep), puis génère les commandes de libération des outils. En cas de REFUS , réinsère la demande dans la file après un délai.

```

1  DWORD WINAPI assembly_thread(LPVOID param) {
2      while (connected) {
3          Response* resp = get_response();
4          if (!resp) continue;
5
6          if (strcmp(resp->reply, REP_OK) == 0) {
7              if (resp->type == 1) {
8                  log_client("ASSEMBLY", "Outil %d alloue,
9                          assemblage...",
10                         resp->id1);
11                  Sleep(2000);
12                  log_client("ASSEMBLY", "Libération outil %d", resp
13                         ->id1);
14                  add_request(CMD_LIBERATION_OUTIL, 1, resp->id1,
15                         -1);
16              } else {
17                  log_client("ASSEMBLY", "Outils %d et %d alloues...
18                         ",

```

```

15         resp->id1, resp->id2);
16         Sleep(3000);
17         add_request(CMD_LIBERATION_OUTIL, 1, resp->id1,
18                     -1);
19         add_request(CMD_LIBERATION_OUTIL, 1, resp->id2,
20                     -1);
21     }
22     } else if (strcmp(resp->reply, REP_REFUS) == 0) {
23         log_client("ASSEMBLY", "Demande refusee, reessai...");
24         Sleep(1000);
25         // Réinsertion de la demande
26         if (resp->type == 1)
27             add_request(CMD_DEMANDE_OUTIL, 1, resp->id1, -1);
28         else
29             add_request(CMD_DEMANDE_DEUX_OUTILS, 2,
30                         resp->id1, resp->id2);
31     }
32     free(resp);
33 }
return 0;
}

```

Listing 5.7: Thread d'assemblage – réaction aux réponses serveur

5.4 Compilation et exécution

```

1 all:
2     gcc serveur/gestionnaire_outils.c -lws2_32 -o serveur.exe
3     gcc client/bras_robotique.c -linclude -lws2_32 -o robot.exe
4
5 run_server:
6     .\serveur.exe
7
8 run_client:
9     .\robot.exe

```

Listing 5.8: Makefile de l'entrepôt intelligent

Procédure d'exécution :

1. Démarrer le serveur : `./serveur.exe`
2. Lancer plusieurs clients (terminaux distincts) : `./robot.exe 1`, `./robot.exe 2`, `./robot.exe 3`...
3. Observer les logs console et le fichier `logs.csv`.
4. Taper Entrée dans chaque client pour l'arrêter proprement.

Chap : 6 Résultats et Analyse

6.1 Extrait du journal de logs

Voici un exemple de contenu du fichier `logs.csv` produit lors d'une session avec deux robots concurrents :

```
1 timestamp;level;message
2 2026-05-05T00:15:20;INFO;Serveur démarre sur le port 8888
3 2026-05-05T00:15:21;INFO;Client 1 connecte
4 2026-05-05T00:15:22;INFO;Client 2 connecte
5 2026-05-05T00:15:23;RECV;Client 1 : DEMANDE_DEUX_OUTILS 0 1
6 2026-05-05T00:15:23;ALLOC;Client 1 a obtenu les deux outils 0 et 1
7 2026-05-05T00:15:23;SEND;Client 1 : OK
8 2026-05-05T00:15:24;RECV;Client 2 : DEMANDE_DEUX_OUTILS 1 0
9 2026-05-05T00:15:24;WOUND;Client 1 blesse client 2 sur l'outil 0
10 2026-05-05T00:15:24;WAIT;Client 2 ne peut pas obtenir les outils
    (0,1)
11 2026-05-05T00:15:24;SEND;Client 2 : REFUS
12 2026-05-05T00:15:26;LIBERE;Client 1 a libere l'outil 0
13 2026-05-05T00:15:26;LIBERE;Client 1 a libere l'outil 1
14 2026-05-05T00:15:27;ALLOC;Client 2 a obtenu les deux outils 0 et 1
```

Listing 6.1: Extrait de logs.csv

6.2 Preuve d'absence de deadlock

Le scénario suivant démontre que les trois mécanismes combinés éliminent tout risque d'interblocage :

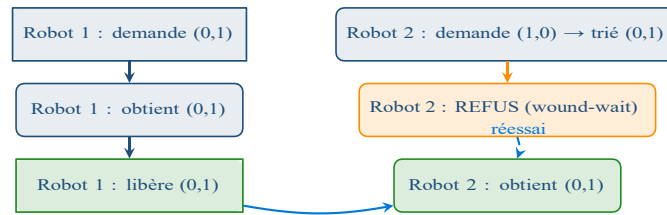


Figure 6.1: Scénario de prévention du deadlock par wound-wait

- **Absence de deadlock** : la hiérarchie des ressources et wound-wait éliminent les cycles d'attente.
- **Vivacité** : tout client âgé finit par obtenir ses ressources (wound-wait garantit le progrès).
- **Sécurité** : le watchdog protège contre les clients défaillants.
- **Traçabilité** : le fichier CSV permet de rejouer et analyser chaque incident *a posteriori*.

Conclusion Générale

Ce mini-projet a permis de mettre en pratique des concepts fondamentaux des systèmes d'exploitation à travers deux réalisations complémentaires.

La **Partie 1** a illustré les différentes politiques d'ordonnancement CPU et leurs compromis : FIFO est simple mais non optimal ; SJF minimise le temps d'attente moyen mais peut engendrer de la famine ; Round Robin garantit l'équité au prix d'une légère dégradation du débit. L'architecture modulaire adoptée, avec les politiques dans des fichiers séparés et un type pointeur de fonction, ouvre la voie à un chargement dynamique des politiques.

La **Partie 2** a abordé des problématiques avancées de programmation système : communication réseau (sockets TCP), concurrence (threads POSIX / Windows), synchronisation (mutex, sections critiques, files d'attente thread-safe) et prévention des interblocages. La combinaison des trois stratégies — hiérarchie fixe, wound-wait et watchdog — fournit une solution robuste et prouvablement exempte de deadlocks.

Parmi les extensions envisageables figurent : l'interface graphique avec ncurses ou GTK pour visualiser en temps réel l'état des outils, l'ajout d'une qualité de service avec file de priorité côté serveur, et le portage Linux complet (remplacement des API Windows par les API POSIX).

FIN .

